Bitácora de defensa

SmartCity Analytics · Avance PostgreSQL + MongoDB · Cesar

Sesión 1 · 13 oct 2026 · Preparar el entorno

Resumen

Dejamos la computadora lista para el proyecto. Se instalaron WSL, Docker Desktop, DBeaver y MongoDB Compass (Python ya estaba), y se conectó el repositorio **SmartCity-Project-CE**. En el repo quedaron creados el **docker-compose.yml**, el **.env.example**, el **.gitignore** y el script **01_ddl.sql** que crea las 18 tablas.

Idea central: Postgres y Mongo no se instalan en Windows. Corren dentro de contenedores de Docker, así los 4 integrantes tienen exactamente el mismo entorno y todo se levanta con un solo comando.

Cómo se conecta todo

```
Tú (PowerShell)  →  docker compose up
                          │
                 Docker Desktop (usa WSL)
                  ├── contenedor Postgres  ←── DBeaver te deja verlo
                  └── contenedor Mongo     ←── Compass te deja verlo
                          ▲
              scripts de Python (llenan las bases con datos)

GitHub Desktop → guarda y comparte todos los archivos con el grupo
```

Herramientas y para qué sirven

Herramienta	Para qué la usamos	Ejemplo sencillo
WSL	Un Linux pequeño dentro de Windows. Docker lo necesita para funcionar.	Una segunda computadora con Linux escondida en la tuya; Docker la usa por debajo.
Docker Desktop	Corre Postgres y Mongo en contenedores, sin instalarlos en Windows.	Imagen = receta del pastel; contenedor = el pastel hecho. Misma receta, mismos pasteles.
Python	Scripts que generan y cargan los datos (100k+ filas, 500k+ documentos).	Un script de 30 líneas crea millones de mediciones en segundos.
DBeaver	Ver y consultar las tablas de PostgreSQL con interfaz gráfica.	Abres la tabla sensor como un Excel o ejecutas SELECT * FROM sensor;
MongoDB Compass	Ver y consultar las colecciones de MongoDB.	Ves cada documento como JSON: { "sensor": "S-00001", "pm25": 18.6 }
GitHub Desktop	Guardar versiones del código y compartirlo con el grupo.	Commit = guardar versión; push = subirla; pull = recibir la de otros.

Archivos del repositorio

Archivo	Para qué sirve	Ejemplo
docker-compose.yml	Lista de contenedores que Docker debe levantar y su configuración.	Postgres 16 en el puerto 5432 y Mongo 7 en el 27017, con un solo docker compose up.
.env.example / .env	Usuarios y contraseñas fuera del código.	El compose usa ${PG_PASSWORD} y Docker lo lee del .env, que nunca se sube.
.gitignore	Lo que git no debe subir.	El .env y los datos generados (pueden pesar GB) se quedan locales.
postgres/init/01_ddl.sql	Crea las 18 tablas de la base.	Docker lo ejecuta solo la primera vez que arranca Postgres.

Comandos clave

Comando	Qué hace
docker --version	Confirma que Docker está instalado.
docker run hello-world	Prueba que el motor de Docker corre (descarga y ejecuta una imagen de prueba).
docker compose up -d	Levanta Postgres y Mongo en segundo plano según el docker-compose.yml.
docker compose ps	Muestra qué contenedores del proyecto están corriendo.
docker compose down -v	Apaga los contenedores y borra los datos (para recrear las tablas).

Preguntas que pueden hacer en la defensa

¿Por qué usar Docker y no instalar PostgreSQL y MongoDB directamente?
Para que los 4 integrantes y el profesor tengan el mismo entorno, con las mismas versiones, y se levante con un comando. Evita el problema de "en mi computadora sí funciona".

¿Cuál es la diferencia entre una imagen y un contenedor?
La imagen es la plantilla (por ejemplo postgres:16). El contenedor es esa plantilla en ejecución. De una imagen se pueden crear varios contenedores iguales.

¿Por qué el archivo .env no se sube a GitHub?
Porque contiene contraseñas. Se sube solo el .env.example como plantilla y cada quien crea su copia local.

¿Cuándo se ejecuta el 01_ddl.sql?
Automáticamente, la primera vez que arranca el contenedor de Postgres con el volumen vacío. Si se cambia el DDL hay que usar docker compose down -v para que se vuelva a ejecutar.

¿Qué es un volumen de Docker?
Un espacio donde Docker guarda los datos fuera del contenedor. Si el contenedor se apaga o se borra, los datos siguen ahí.

Concepto clave · MongoDB y cómo se reparte el trabajo con PostgreSQL

¿Qué es MongoDB?

MongoDB es una base de datos **NoSQL de documentos**. En vez de guardar la información en tablas con filas y columnas, la guarda como **documentos JSON** dentro de **colecciones**.

PostgreSQL	MongoDB
Base de datos	Base de datos
Tabla	Colección
Fila	Documento (un JSON)
Columna	Campo
Todas las filas tienen las mismas columnas	Cada documento puede tener campos distintos

Ejemplo: dos sensores que miden cosas distintas

```
{ "sensor": "S-00001", "tipo": "calidad_aire",
  "ubicacion": { "zona": "Centro", "lat": 9.93, "lon": -84.08 },
  "medicion": { "pm25": 18.6, "co2": 425 },
  "timestamp": "2026-10-13T14:00:00" }

{ "sensor": "S-00002", "tipo": "trafico",
  "ubicacion": { "zona": "Norte", "lat": 9.95, "lon": -84.10 },
  "medicion": { "vehiculos": 132, "velocidad_prom": 34.5 },
  "timestamp": "2026-10-13T14:00:00" }
```

El primero tiene **pm25** y **co2**; el segundo, **vehiculos** y **velocidad_prom**. En Mongo conviven en la misma colección. En PostgreSQL habría que crear una columna por cada medición posible, la mayoría quedaría vacía y cada sensor nuevo obligaría a modificar la tabla.

Por qué lo usamos en el proyecto

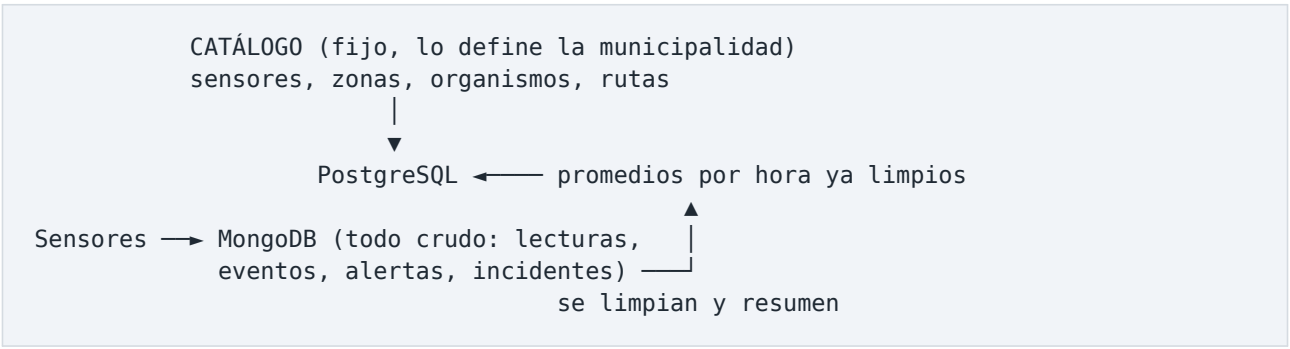
Los sensores IoT generan datos **semiestructurados**, en mucho volumen y muy seguido. Mongo encaja porque: (1) tiene **esquema flexible**, un sensor nuevo entra sin cambiar nada; (2) la ubicación va **embebida** dentro de la lectura, así que no hace falta JOIN; (3) es rápido para escribir grandes volúmenes (el proyecto pide 500.000+ documentos).

Cómo se consulta: Aggregation Framework

```
db.lecturas.aggregate([
  { $group: { _id: "$ubicacion.zona",
              promedio: { $avg: "$medicion.temperatura" } } }
])
```

Es una tubería de pasos por la que pasan los documentos: **$match** filtra, **$group** agrupa, **$sort** ordena. El ejemplo calcula la temperatura promedio por zona; equivale a un GROUP BY en SQL.

Cómo se reparten el trabajo MongoDB y PostgreSQL



	MongoDB	PostgreSQL
Qué guarda	Lo que mandan los sensores tal cual: lecturas, eventos, alertas, incidentes	El catálogo (sensores, zonas, organismos, rutas, vehículos) y los promedios por hora ya limpios
De dónde viene	De los sensores	El catálogo lo define la municipalidad; los promedios salen de los datos crudos
Qué aporta	Flexibilidad y volumen	Integridad, reglas y relaciones entre tablas
Se consulta	Sí: sensores sobre el límite, alertas por zona, etc. (consultas de D)	Sí: sensores por tipo, ranking de zonas, etc. (consultas de B y C)

El vínculo entre las dos bases es el id_sensor (por ejemplo S-00001): es el mismo código en ambas, así una lectura de Mongo se puede cruzar con los datos del sensor en Postgres.

En el proyecto completo, el paso de limpiar y resumir lo hará **Spark** (próximo avance). En este avance, los scripts de Python llenan las dos bases directamente.

Preguntas que pueden hacer en la defensa

¿Por qué usar MongoDB aquí?
No porque guarde JSON, sino porque las lecturas de sensores son semiestructuradas: cada tipo de sensor tiene campos distintos, y un modelo documental las guarda sin forzar un esquema rígido.

¿Por qué no guardar todo en MongoDB o todo en PostgreSQL?
Mongo guarda lo que llega de los sensores tal cual, con su flexibilidad. Postgres guarda el catálogo y los datos ya limpios, con integridad y relaciones. Cada uno se usa donde es más fuerte; el id_sensor las une.

¿Qué significa embeber un documento?
Guardar un dato dentro de otro documento en vez de en otra tabla. Por ejemplo, la ubicación dentro de la lectura: se lee todo junto sin JOIN.

¿Qué es el Aggregation Framework?
La forma de hacer análisis en Mongo: una tubería de etapas ($match, $group, $sort...) que transforma los documentos paso a paso, como un GROUP BY de SQL.